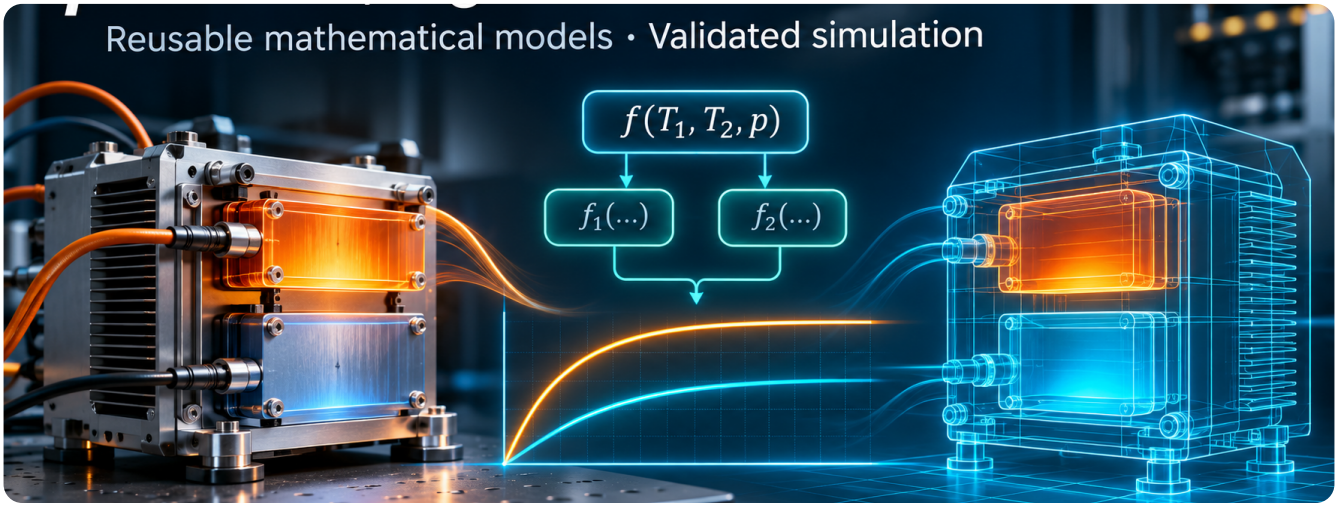
PACEVAL UG · SCIENTIFIC RESEARCH DEMONSTRATOR

Programmable Mathematical Execution for Digital Twins

A validated, reusable thermal-model mathematics layer with nonlinear scenarios and independent numerical checks



Research result. The proposed digital-twin mathematics use case was implemented and executed with the original paceval Linux runtime. Explicit thermal functions were prepared once and reused for batch evaluations, coupled-state time integration and five parameter variations. The host retained ownership of integration, scenario parameters and state bookkeeping.

32

Full two-zone scenarios

180 integration steps each; synthetic and deterministic

5 + 3

Reusable paceval graphs

Five ordinary expressions and three TINC-enabled evaluations

Scientific scope. The demonstration tests the **mathematical feasibility** of an auxiliary execution layer, not the accuracy of a physical production asset. No external machine measurements were used; no FMU was generated, and no general runtime or hardware advantage is claimed.

1. Research context and integration boundary

Digital twins combine physical-model definitions, measured or estimated parameters, state integration and application-level decisions. The attached 4 November 2026 research proposal identified a specific opportunity for paceval: repeated execution of explicit algebraic functions without replacing established time integrators. The Functional Mock-up Interface (FMI) 3.0.2 specifies Model Exchange as a host-integrated formulation in which the importer provides numerical time integration. This study follows that separation architecturally, without claiming to implement the FMI API.

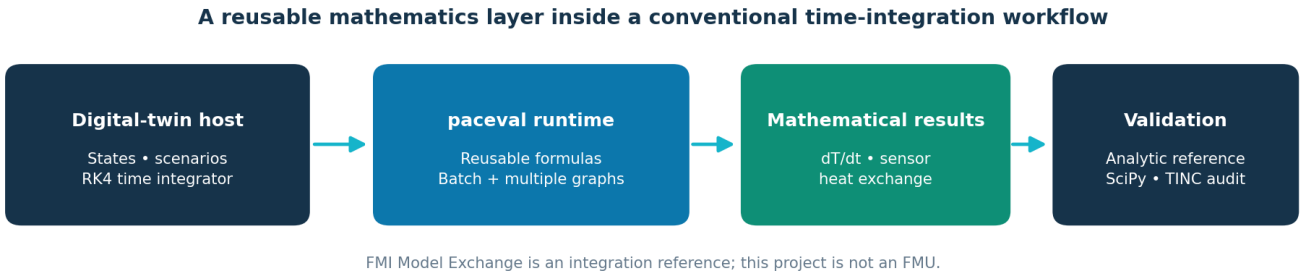


Figure 1. The C++ host advances the simulated state; the real paceval runtime evaluates prepared mathematical expressions; independent references assess results. FMI is an architectural reference, not a delivered FMU.

The material contribution of this demonstrator is concrete: an exact single-state reference, a two-state nonlinear extension, interchangeable native and paceval derivative backends, sensor and heat-loss outputs, a parameter-sweep workflow, and an explicitly recorded verification protocol.

2. Work package A: exact one-state thermal reference

For a thermal capacitance $C_1 > 0$, conductance $h_1 > 0$, input power P and fixed ambient temperature T_a, the first-order thermal balance is:

$$C_1 \cdot dT_1/dt = P - h_1(T_1 - T_a)$$
$$f_0 = [P - h_1(T_1 - T_a)] / C_1$$
$$T_1(t) = T_a + P/h_1 + [T_1(0) - T_a - P/h_1] \cdot \exp(-h_1 t/C_1)$$

This model is deliberately small because its closed-form solution gives an independent test oracle. The paceval graph represents f_0 and is evaluated repeatedly as a C++ RK4 integration proceeds. The host's numerical trajectory matched the exact model solution within the documented tolerance.

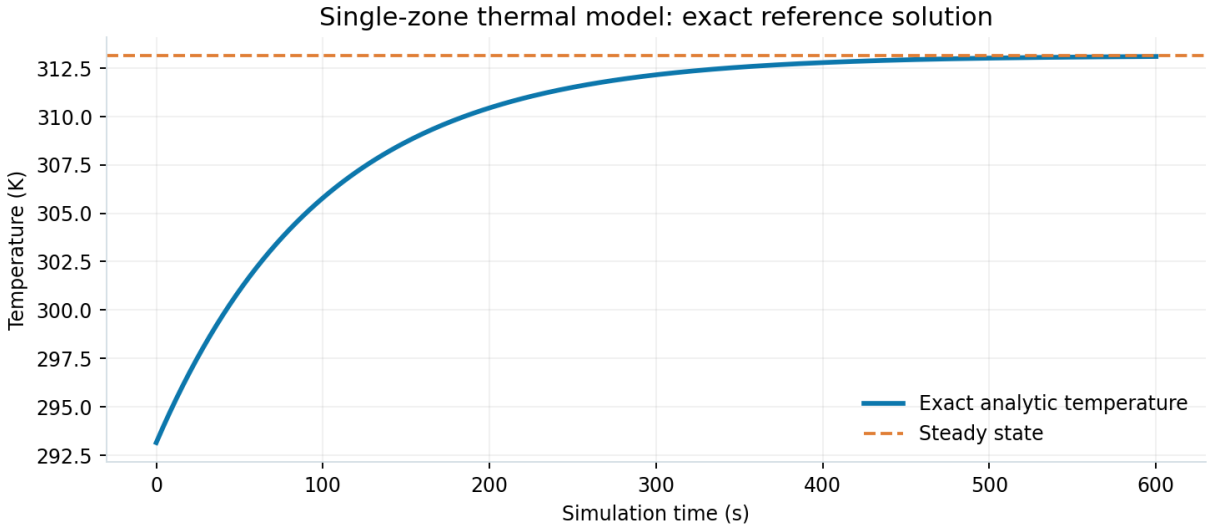


Figure 2. Analytic one-zone temperature solution with its steady state. Illustrative physical parameters, not measured plant data.

3. Work package B: nonlinear coupled-state digital twin

The expanded model represents two coupled temperature states. The first compartment receives input power, while a second compartment exchanges heat with the first and the environment. A fourth-power

term is deliberately included as a nonlinear surrogate to exercise reusable expression graphs beyond an elementary linear model.

$$C_1 \cdot dT_1/dt = P - k_{12}(T_1 - T_2) - h_1(T_1 - T_a)$$
$$C_2 \cdot dT_2/dt = k_{12}(T_1 - T_2) - h_2(T_2 - T_a) - Q_{nl}$$
$$Q_{nl} = k_{rad}[(T_2 / 300\ K)^4 - (T_a / 300\ K)^4]$$
$$y(t) = gain \cdot T_2(t) + offset$$

All temperatures are in kelvin; **C_1,C_2** are in J/K; **h_1,h_2,k_{12}** are in W/K; **P** and **k_{rad}** are in W. The nonlinear term is a documented **synthetic surrogate**, not a fitted radiative transfer law or independently validated physical device model.

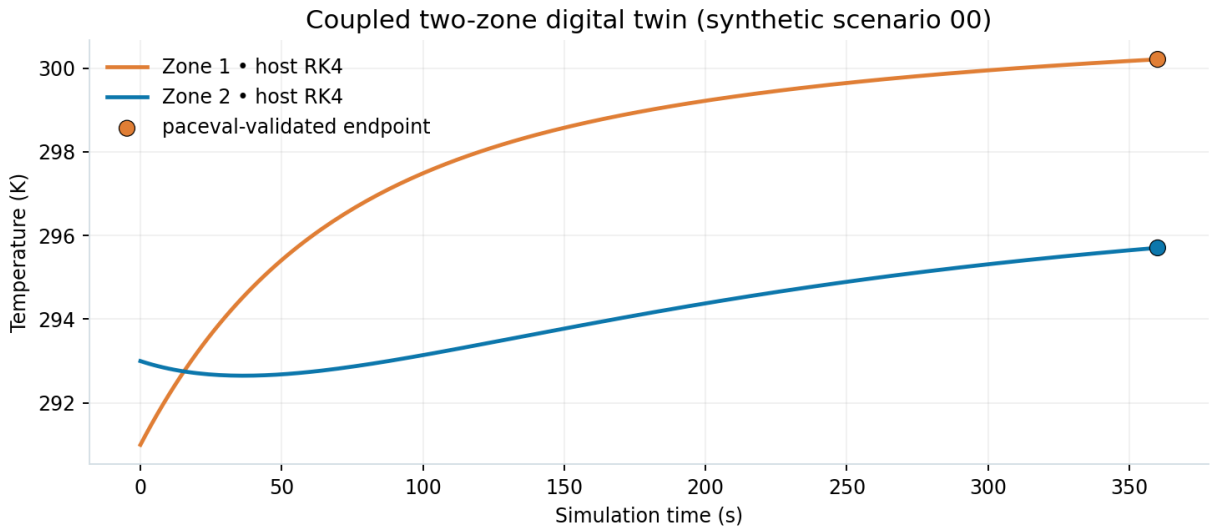


Figure 3. Coupled thermal-state trajectories for one deterministic scenario. The final paceval-computed states agree with the native C++ integration; both use the same RK4 host.

Energy conservation is checked explicitly

The coupling heat flux cancels in the summed energy balance. For the implemented right-hand sides:

$$C_1\, dT_1/dt + C_2\, dT_2/dt = P - h_1(T_1 - T_a) - h_2(T_2 - T_a) - Q_{nl}.$$

The code recomputes the right-hand-side identity at the supplied inputs and checks the residual independently of the graph implementation. This is a physical-consistency test of the chosen equations, not validation of their suitability for a particular machine.

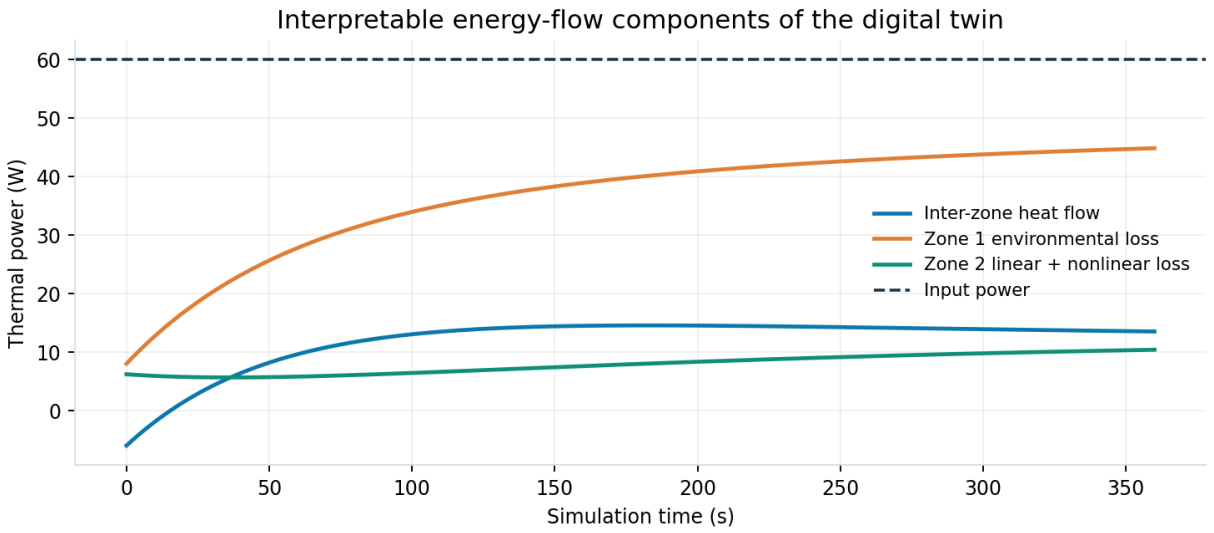


Figure 4. Time-dependent heat flow between compartments and to the environment, compared with imposed thermal power.

4. Exact paceval implementation

With the actual original Linux shared library, the C++ demonstration now uses the official paceval SDK C/C++ interface directly. The application includes `src/paceval_main.h`, and CMake compiles the supplied `src/paceval_main.cpp` together with `src/digital_twin.cpp`, linking to the separately provided `libpaceval_linux_sharedLIB.so`. The API wrapper forwards the typed, documented `paceval_*` functions to the library's exported entry points. It creates ordinary computation objects once, reuses them over different temperatures and physical parameters, and invokes single- or multi-graph batch evaluation.

Prepared graph	Mathematical responsibility
One-state derivative	Constant-input first-order thermal right-hand side; analytic reference exists
Two-zone derivative 1	Input power, inter-zone coupling and first-zone heat loss
Two-zone derivative 2	Inter-zone coupling and linear/nonlinear second-zone heat loss
Virtual sensor output	Explicit affine transformation of temperature state T_2
Second-zone heat loss	Explicit linear + fourth-power surrogate output for audit

Three additional graphs enable TINC for the one-state derivative and both two-state derivatives. Source file `src/digital_twin.cpp` calls `paceval_InitializeLibrary`, `paceval_CreateComputation`, `paceval_dGetComputationResultExt`, `paceval_dGetMultipleComputationsResultsExt`, `paceval_dGetComputationResult`, `paceval_GetIsError`, `paceval_GetErrorInformation`, `paceval_DeleteComputation` and `paceval_FreeLibrary`. Handle creation and deletion are managed with an exception-safe C++ RAII wrapper. The 12 variables use the same declaration order for every graph, enabling mixed multi-graph evaluation.

Independent host responsibilities: fourth-order Runge–Kutta integration, input validation, model time, state propagation, scenario construction, CSV I/O, balance checks and reference comparisons. There is no generated or imported FMU. This boundary is intentional for a precise proof of mathematical integration feasibility.

Reproducible Linux build. Provide the original shared library at configure time using the absolute CMake variable `PACEVAL_RUNTIME` (or environment variable `PACEVAL_LIB`). Build with `cmake -S src -B build`, then `cmake --build build -j` and `ctest --test-dir build --output-on-failure`. The official SDK API layer is compiled directly; the vendor binary is linked but never included in the GitHub distribution. See `README.md` for the full commands.

5. Reused model equations across parameter studies

With the ordinary paceval expression graphs unchanged, the host produces five synthetic input variations of power and thermal-loss parameters. Each variation receives the same integration logic and distinct physical inputs. A parameterized graph, rather than a freshly generated formula for each variation, supplies the derivative values.

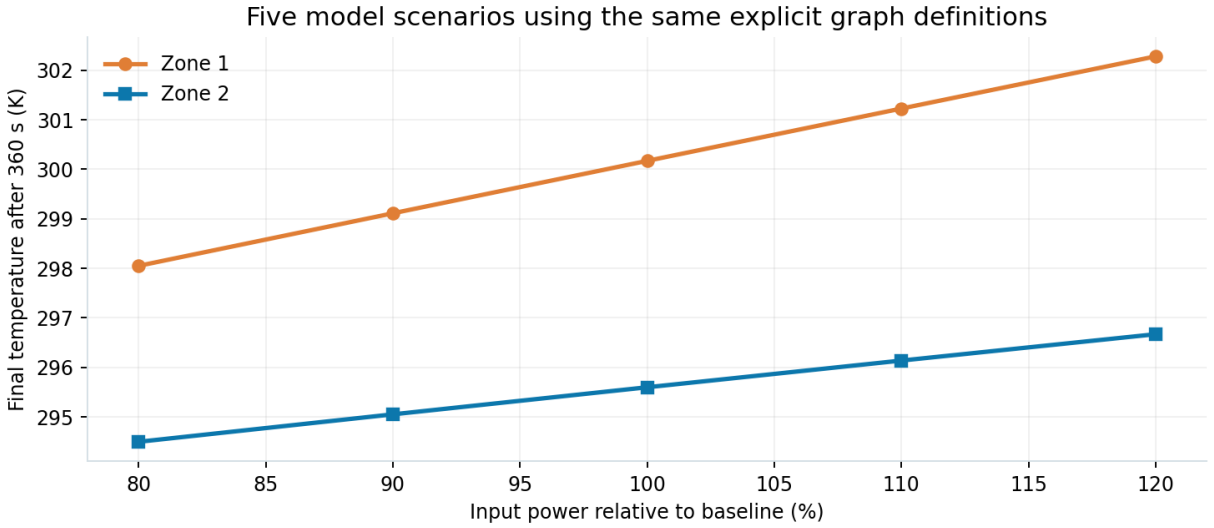


Figure 5. Five synthetic power-scenario end states demonstrate explicit model reuse across input conditions. The plot is a physical response illustration, not a speed benchmark.

6. Reproducible numerical validation

The demonstration includes multiple **independent** correctness checks rather than relying only on agreement between graph functions evaluated by the same runtime. The test platform was Debian GNU/Linux 13 (x86-64, virtualized AMD EPYC 9V74), GCC 14.2 and Clang 17, with C++17, the original paceval Linux shared library and the two provided paceval API source files. Compilation and linking were checked using both compilers and the same external runtime.

Validation method	Observed result
Native unit tests	Analytic single-zone solution, input ranges, energy balance and RK4 step-halving checks passed
Native and paceval C++ integration	32 scenarios × 180 steps (2 s/step), five parameter variations, 579 numerical comparisons; zero configured tolerance violations
Independent adaptive ODE reference	SciPy DOP853 reference on 32 final trajectories, with maximum absolute deviation about 2.016×10^{-9} K
Reproducible build	Both registered CTest tests passed with GCC and Clang using the original paceval shared library

Pointwise TINC audit	24 finite, ordered returned intervals; 22 included the 110-digit exact-input reference, 2 did not

The independent high-precision calculation uses `mpmath` at 110 decimal digits, reconstructing each exact binary64 input value before evaluation. The test package includes `src/high_precision_reference.py`, `src/validate_trajectories.py` and the original reproducible CSV inputs and outputs in `examples/`.

> **Numerical interval qualification.** Two of 24 pointwise TINC outputs were degenerate (lower bound equals upper bound) and did not enclose the independently evaluated mathematical result. The corresponding one-zone and first-zone derivative examples are preserved in `examples/tinc_samples.csv`. Consequently, this study demonstrates how TINC outputs can be collected and audited but does **not** establish a general rigorous enclosure guarantee. The high-precision checker intentionally returns an error for these non-enclosures.

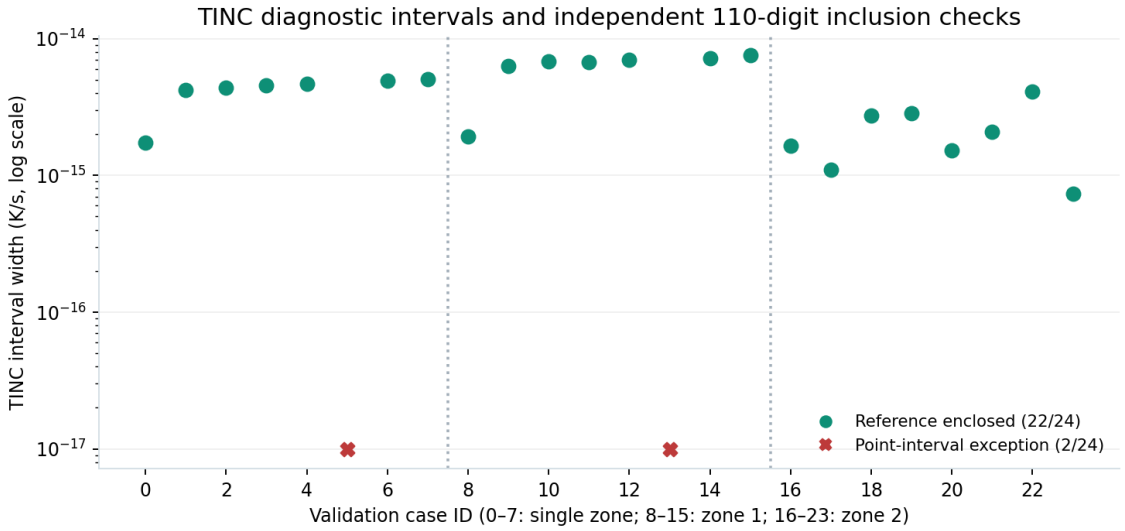


Figure 6. TINC widths and independent inclusion assessment. Green = sampled inclusion; red X = the two point-valued intervals not containing the reference. The chart is an audit rather than a proof of general interval validity.

7. Integration outlook and scientific conclusion

The combined result is a **successful software feasibility demonstration** of paceval as a reusable explicit-mathematics component for selected digital-twin submodels. It establishes the C/C++ integration pattern, independent numerical agreement of thermal trajectories, reuse of graph handles across multiple scenarios and a transparent interval-audit workflow.

A natural research continuation would export the same mathematical interface through a genuine FMI Model Exchange FMU, connect real thermal sensors, identify parameters against calibration data, and validate event handling, uncertain measurements and external numerical-solvers. Programmable FPGA/ASIC implementations of prepared mathematical graphs may also be studied separately. These extensions were **not** implemented or measured in this study.

GitHub deliverables and exact references

`README.md` (documentation and reproducer); `src/digital_twin.cpp` (documented mathematical host); `src/paceval_main.h` and `src/paceval_main.cpp` (user-supplied SDK interface); `src/CMakeLists.txt` (link to external library, build and CTest); `src/high_precision_reference.py` (TINC audit); `src/validate_trajectories.py` (independent adaptive solver); `src/make_figures.py` (figure generation); `examples/scenarios.csv`, `examples/trajectories.csv` and `examples/tinc_samples.csv` (reproducibility); `figures/` (cover and Figures 1–6). All filenames refer to the delivered repository. The SDK sources retain their GNU Affero General Public License

notice. For this stand-alone source layout, the `paceval_main.cpp` header include path was adjusted to `"paceval_main.h"`; the vendor files were converted from Windows-1252 to UTF-8 without changing the SDK's function definitions.

Source material and supporting references

[1] paceval UG (4 November 2026), *Programmable Mathematical Execution for Digital Twins*, user-provided scientific research proposal (the conceptual basis of this expanded demonstrator).

[2] Modelica Association Project FMI, *Functional Mock-up Interface 3.0.2*, 27 November 2024. https://fmi-standard.org/docs/3.0.2/

[3] paceval UG, *paceval mathematical engine, C/C++ API and integration instructions*. https://paceval.com/api/

[4] paceval UG, *Trusted Interval Computing (TINC)*. https://paceval.com/tinc/

[5] SciPy, *solve_ivp, DOP853 numerical integration*. Official documentation

[6] paceval UG, *paceval SDK 4.25*, `paceval_main.h` and `paceval_main.cpp` (supplied for this project); source header states GNU Affero General Public License. GitHub overview.

Research contact: paceval UG, Munich, Germany · info@paceval.com · https://paceval.com · This report is a synthetic software feasibility study, not independent endorsement by a machine owner or the FMI standards organization.